

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 3.1: Konfigurationsmanagement

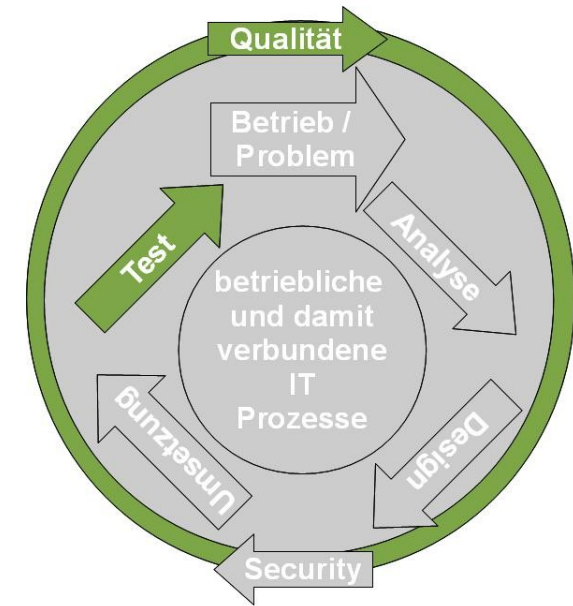
v. 09.01.2016

Einordnung Konfigurationsmanagement

Softwarekonstruktion
WS 2016/17



- Modellgetriebene SW-Entwicklung
- Qualitätsmanagement
- Testen
 - Einführung
 - Grundlagen Softwaretesten
 - Testen im Softwarelebenszyklus
 - Statischer Test
 - Softwaremetriken
 - Black-Box-Test
 - White-Box-Test
- Konfigurationsmanagement



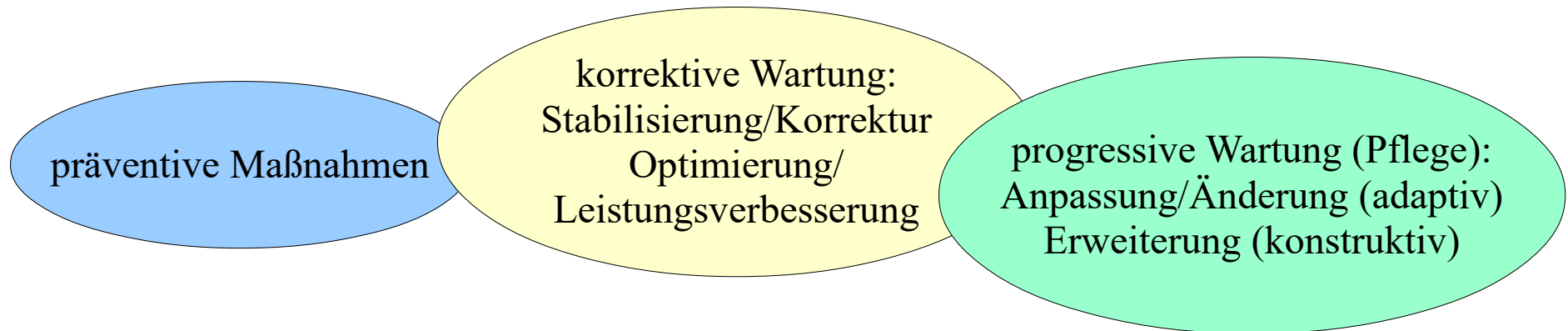
Literatur:

- Humble, J., Farley, D.: Continuous delivery: reliable software releases through build, test, and deployment automation. Addison-Wesley

- Entwicklung eines Verständnis von dem Objekt Programmcode und ähnlicher Artefakte
- Konstruktionspipeline von Transformatoren und Compilern ausgehend von Code in ein oder mehrere Produkte
- Übersicht entwickeln über verschiedene Quellen für Produkte

- Konfigurationsmanagement
 - Vorgehensmodelle KM
- Dateibaumbasierte KM Werkzeuge
 - SVN
 - GIT
- Build Prozesse

In der Wartungs- & Pflegephase lassen sich durchzuführende Aktivitäten in folgende Gruppen einteilen:



Wartungsanforderungen bzgl. Dringlichkeit:

- sofort (operativ): schwerwiegende SW-Fehler, die weitere Nutzung in Frage stellen
- kurzfristig: Code-Korrekturen
- mittelfristig: Systemerweiterungen (upgrades)
- langfristig: Restrukturierung (System Redesign)

- Gesetz der kontinuierlichen Veränderung (Lehmanns 1. Gesetz)
 - Kartoffeltheorem: Ein genutztes System verändert sich, bis die Neustrukturierung oder eine neue Version günstiger ist.
- Gesetz der zunehmenden Komplexität (Lehmanns 2. Gesetz)
 - Ohne Gegenmaßnahmen führen Veränderungen zu zunehmender Komplexität.
- Gesetz des Feedbacks (Lehmanns 3. Gesetz)
 - Die Evolution eines Systems wird immer durch einen Rückkopplungsprozess (feedback process) gesteuert.

- Gesetz der System-Evolution
 - Die Wachstumsrate globaler Systemattribute (z. B. LOC, Anzahl Module) im Laufe der Zeit erscheint stochastisch, sie folgt jedoch einem statistisch bestimmten Trend.
- Gesetz der Grenze des Wachstums
 - Ab einem gewissen Grad an Veränderungen eines Systems treten Probleme bzgl. Qualität und Verwendbarkeit auf.

Eine SW-Konfiguration ist die **Gesamtheit der Artefakte** (Menge von Komponenten in zeitlichen Versionen und funktionalen Varianten), die **zu einem bestimmten Zeitpunkt des Life Cycle** in ihrer Wirkungsweise und ihren Schnittstellen aufeinander abgestimmt sind.

Unter **SW-Konfigurationsmanagement** versteht man die Gesamtheit der Methoden, Werkzeuge und Hilfsmittel, die die Entwicklung, Wartung und Pflege eines Softwareprodukts durch die Konfiguration geeigneter Varianten in passenden Revisionen unterstützt.

Ziele

- Bestimmung der Artefakte (Komponenten, Module, Pakete, Dateien, Datenbanken), die eine Konfiguration bilden; Ableitung aus der PBS
- Konsistenzsicherung, Sichtbarkeit, Verfolgbarkeit, Kontrollierbarkeit von Produkten

Langfristige Ziele über Jahre hinweg

- Prüfung, ob Konsistenz der SW-Konfiguration erhalten bleibt
- Sicherstellung, dass jederzeit, jahrzehntelang auf vorherige Konfigurationen zurückgegriffen werden kann
- Erfassung und Nachweis aller Änderungen; Überwachung der (Produkt-)Konfigurationen während des Lebenszyklus
- Auslieferungskontrolle

- Der Gegenstand des KM ist das gesamte Softwaresystem mit seinen Komponenten und Artefakten (Produkt- und Artefaktstruktur)
 - Spezifikation: Daten und Requirements (Anforderungen)
 - Entwurf: formale und informale Dokumente
 - Programme: Code-Teile, Datenbeschreibungen, Prozeduren
 - Testfälle und -umgebung: Dokumente für Testdaten, Testumgebung
 - Integrationskonzept: alle Dokumente für Integration und Einführung (auch Benutzerdokumentation)
- Verwaltung der Komponenten in der Produkt-(Artefakt-)bibliothek

- **Konfiguration:** benannte und formal freigegebene Menge von Komponenten für eine Abarbeitung (Editierung, Compilation, Link oder GO-Lauf)
- **Version (Revision):** Instanz bzw. Entwicklungsstand einer Komponente; je Editierung erhöht sich die Versions-Nr. um eins;
 - Identifikation: (Komponenten-Name, Versions-Nr., Variantenkennzeichnung)
- **Komponente:** = Software-Artefakt, Baustein
- **VOB** Versioned Object Base sind die Repräsentation von Entitäten (Objekte mit Funktionalität) des Systems in der Mini-Welt des Anwendungsprojektes

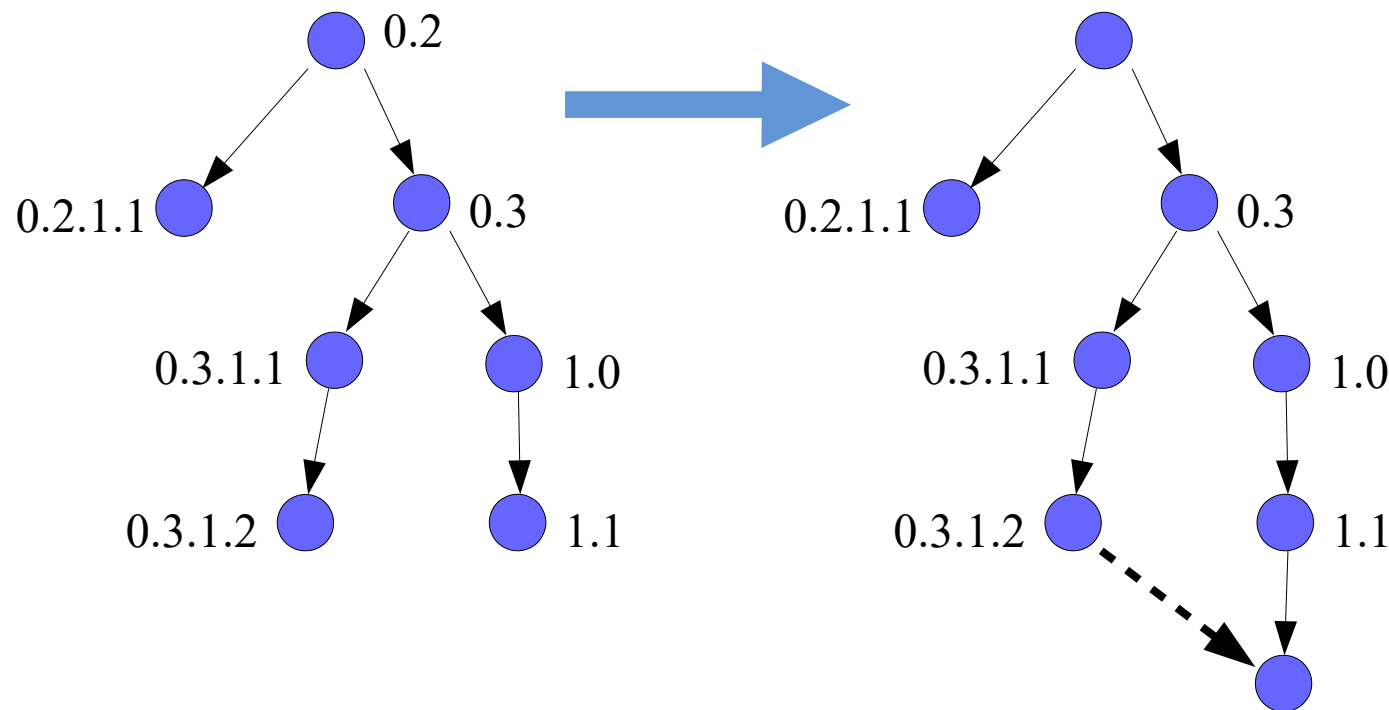
- **Variante:** weitere Untergliederungskordinate für Änderungsstände, Modifikationen, Revisionen innerhalb einer Version der Komponente (z.B. für andere Umgebung)
- **Release:** eine für die Nutzung freigegebene Version (konsistente Menge von VOB)
- **Baseline** (Stückliste) oder **Tag:** gesichertes Zwischenergebnis (Entwicklungsstand), das aus Menge freigegebener Versionen einer Komponentenmenge besteht.
- **Spezifikation:** angepasste Konfigurationen für ein Projekt, freigegeben für ein spezifisches Anwendungsprofil oder einen Auftraggeber

- **Checkout:** Lesen eines Quelltextes der Version n für Editierung
- **Checkin:** Schreiben eines Quelltextes der Version $n+1$ nach Editierung
- **Re-Konfigurierung:** Aktualisierung einer existierenden Konfiguration bei neuen Versionen oder Varianten mit dem Ziel der Konsistenz
- **Branch (Zweig):** Eine Sequenz von Versionen über eine Zeitlinie
- **CI (configuration item):** Element einer Konfiguration

Def: Der Versionsverwaltung obliegt die persistente Verwaltung der Komponenten.

- Identifizierung von Versionen: Komponentennamen, Versions-Nr., Variante
 - Komponentennamen können Pfadnamen, Surrogates oder URLs sein
- Festlegung aller zu verwaltender Bestandteile, die zu einer Komponente gehören, wie Analyse-Spezifikation,
 - Entwurfsspezifikation, Code der Komponente, Testspezifikation, Dokumentation, ...
- Bestimmen der Namenskonventionen und Bezeichnung der Relationen zwischen den Komponenten
- Abspeichern verschiedener Versionen einer Komponente einschließlich ihrer Wiederauffindungspfade
- Bereitstellen beliebiger abgelegter Versionen
- Dokumentieren von Änderungen
- Festlegen und Kontrollieren von Zugriffsrechten
- Verwalten des Komponenten-Repositories

- Verzweigungsbaum unbestimmter Tiefe und Breite
- Jede Modifikation generiert ein Kind oder einen Zweig (branch)
- Verschmelzen ist möglich (branch merge)



- MUE.0.0.1:RELEASE — Alpha test release
- MUE.1.0.0:RELEASE — Erstes Hauptrelease
- MUE.1.2.1:RELEASE — Zweites kleineres Release mit Bugfixes
- MUE.2.0.3:RELEASE — Zweites Hauptrelease mit drei Folgen von Bugfixes

Drei-Nummern Versionsidentifikation

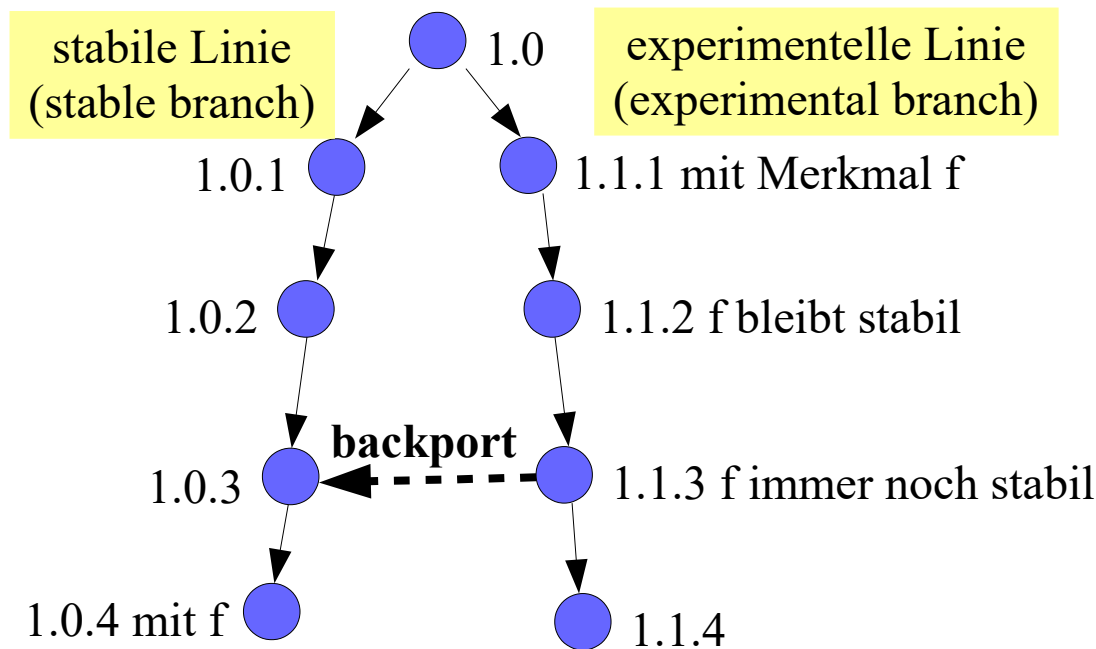
$\langle \text{Version} \rangle ::= \langle \text{CI Name} \rangle . \langle \text{Haupt} \rangle . \langle \text{Minor} \rangle . \langle \text{Revision} \rangle$

$\langle \text{Haupt} \rangle ::= \langle \text{nichtnegative ganze Zahl} \rangle$

$\langle \text{Minor} \rangle ::= \langle \text{nichtnegative ganze Zahl} \rangle$

$\langle \text{Revision} \rangle ::= \langle \text{nichtnegative ganze Zahl} \rangle$

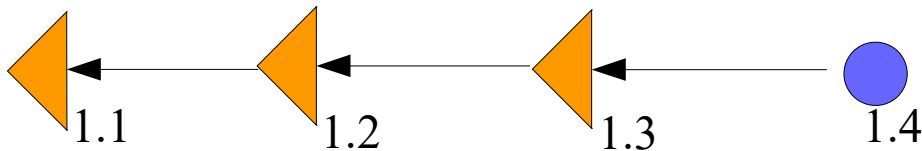
- Stabile Versionen erhalten oft eine „gerade Linie“ von Versionsnummern, experimentelle eine ungerade Linie
- Erweist sich ein Feature auf einer experimentellen Linie als stabil, dann es in die stabile Linie **rückintegriert werden (backporting)**
- Vorteil: stabile Linien erlauben es, kommerzielle Nutzer zu befriedigen, Backporting frischt alte stabile Versionen auf



- Vollständige Kopie jeder Version
- Delta (Differenzen zwischen zwei Versionen)
- Vorwärtsdelta (forward delta)



- Rückwärtsdelta (reverse delta)



- Gemischte Delta (mixed delta)

- Hauptproblem der verteilten Kollaboration

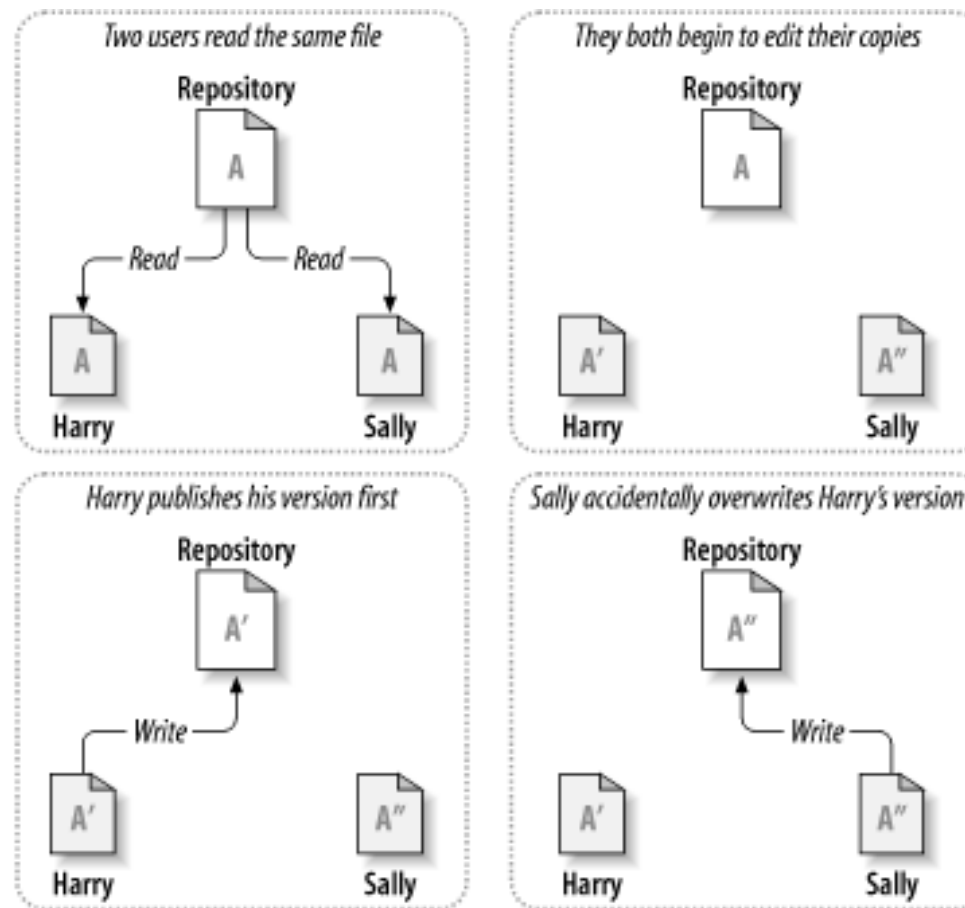


Bild aus svn-book

- Pessimistisches Modell: lock-modify-unlock

Probleme:

- Vergessen zu entsperren
- Paralleles Arbeiten unmöglich
- Deadlock Situation

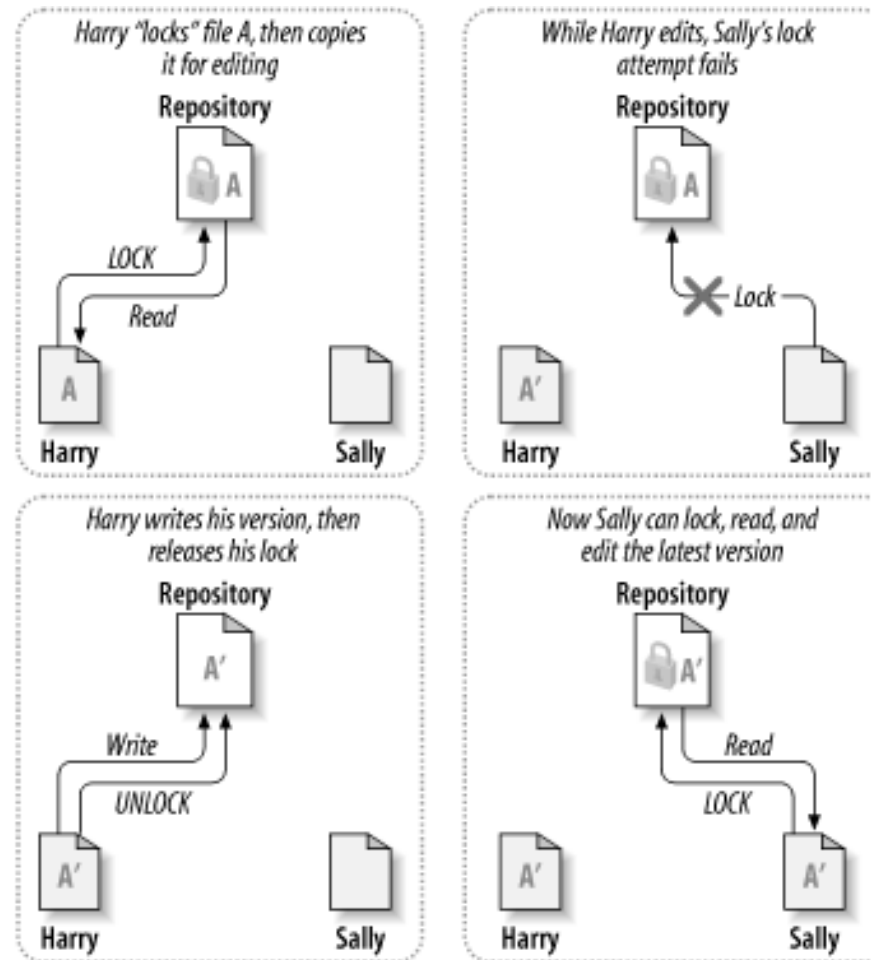


Bild aus svn-book

- Optimistisches Modell: copy-modify-merge

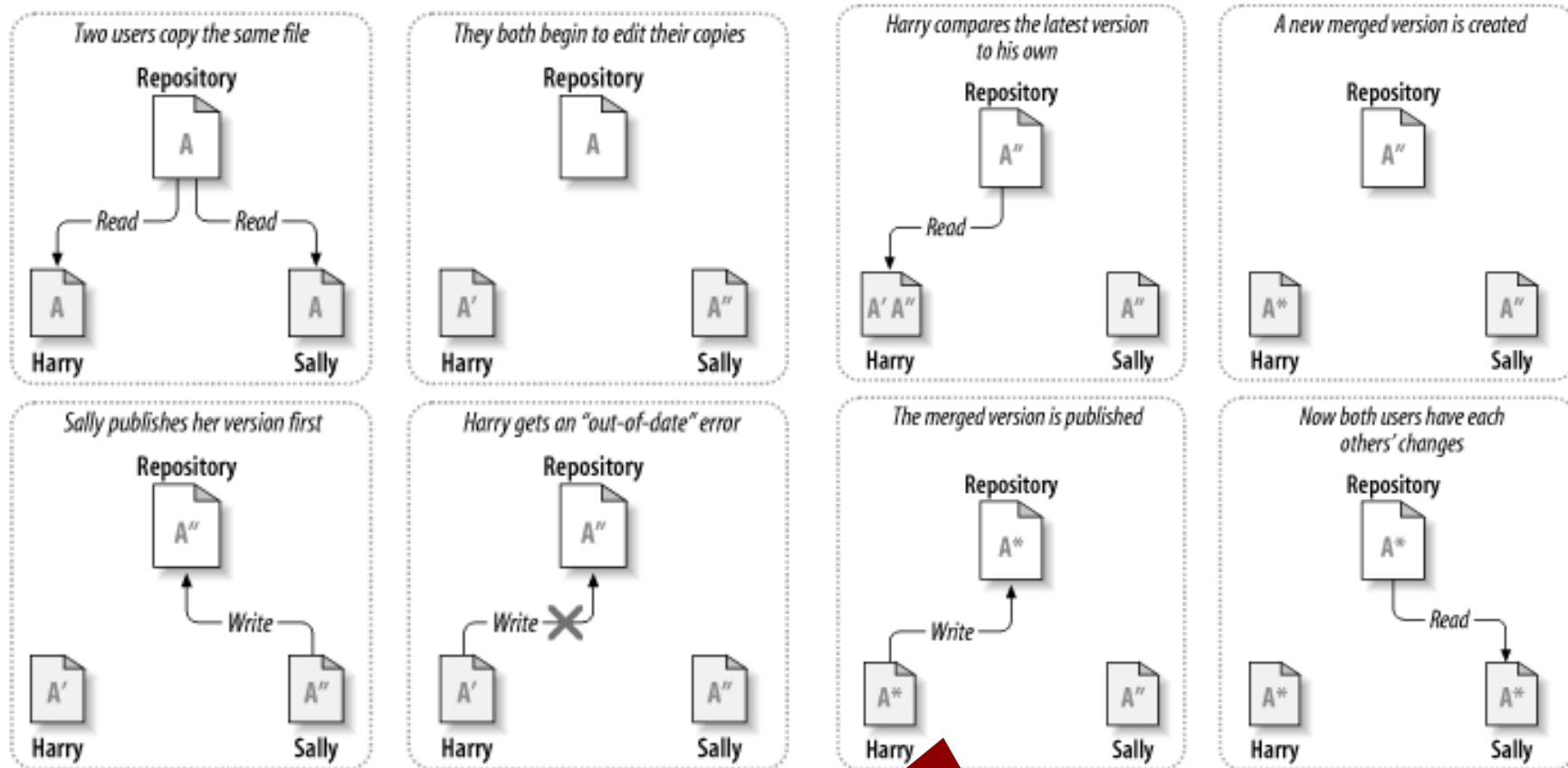
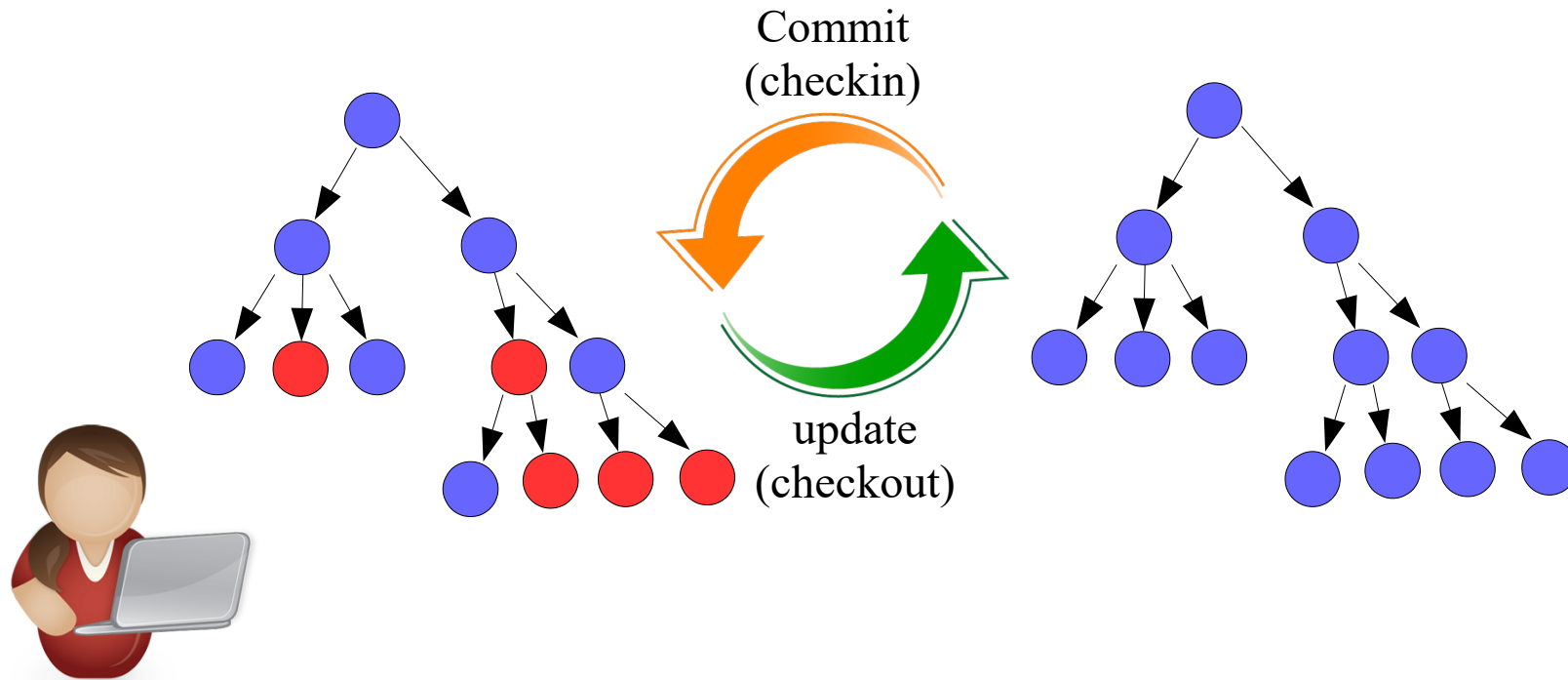


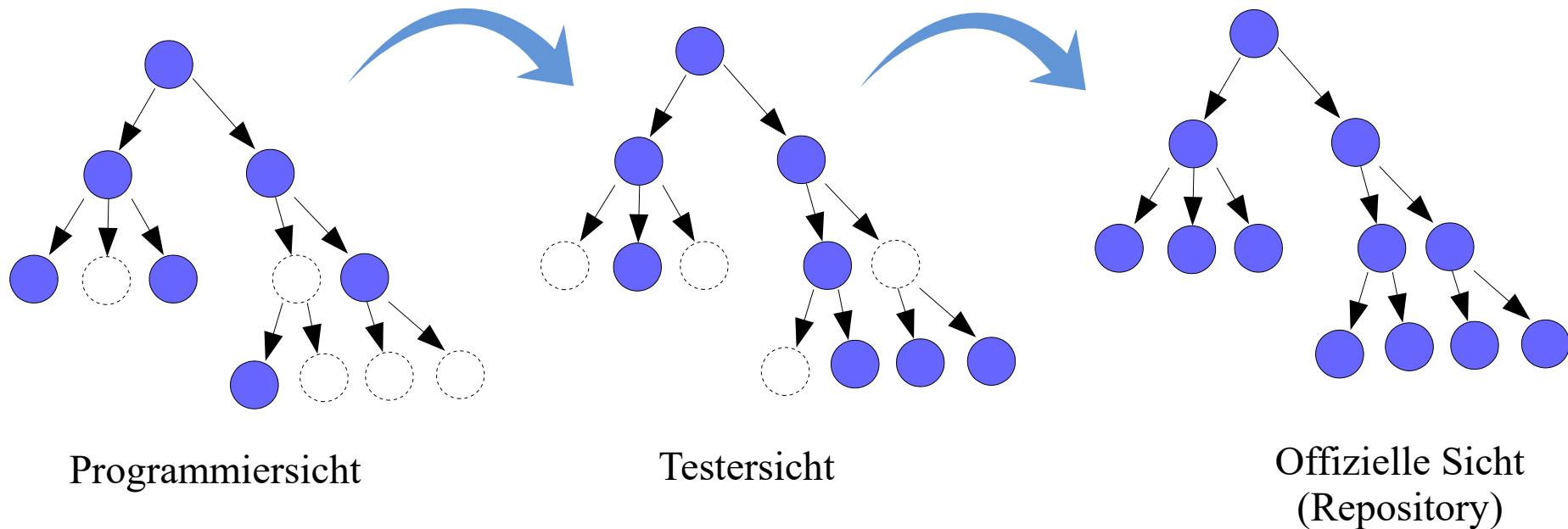
Bild aus svn-book

- Bei paralleler Bearbeitung werden Sichten inkonsistent
- Inkonsistente optimistische Überlappungen des Codes

Nur Tiefe von 1 (CVS und SVN)

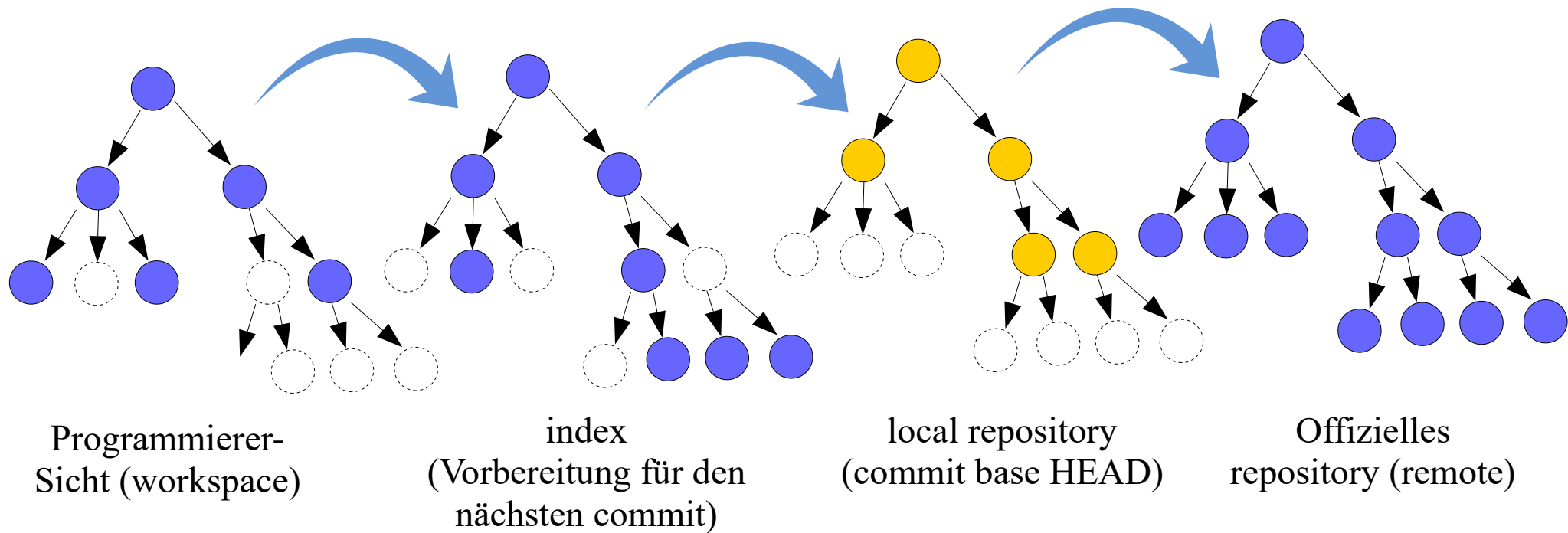


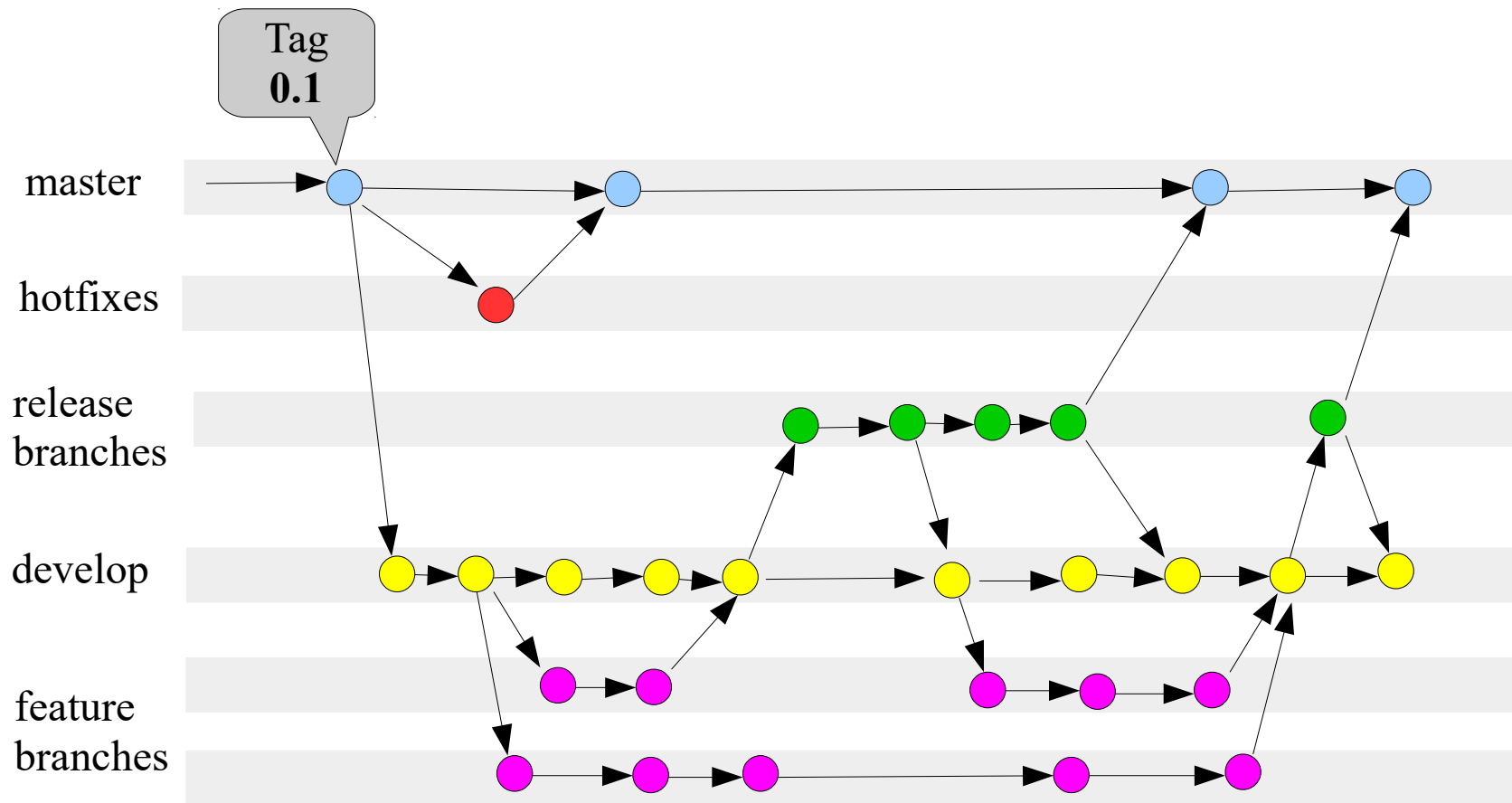
- Programmierer haben eine Sicht von der Testversion, die eine Sicht von der offiziellen Version ist
- Sichten werden Zweige (branches) genannt, wenn sie über mehrere Versionen hinweg parallel entwickelt werden



- Ein zentrales Repository ist keine Pflicht, sondern beliebig “transportierbar”
- Statt dessen GIT workflows, verschiedene Arten, wie Sichten zusammenarbeiten
- GIT speichert keine Deltas, sondern Vollversionen
- GIT kennt features, die benannten Branches entsprechen

Programmierer haben eine Sicht vom workspace, die eine Sicht von der Sicht von der Sicht der offiziellen Version ist





- **Ziel:** Erzeugung und Bereitstellung von neuen Softwareversionen für die Öffentlichkeit
- **Formate**
 - Source Code + Build Skripte + Instruktionen
 - Pakete von ausführbaren Dateien für spezifische Plattformen
 - Andere portable Formate: Java Web Start, plugins
 - Patches und Updates: automatisch oder manuell
- **Inhalt**
 - Source und/oder binär Dateien, Datendateien, Installationsskripte, Bibliotheken, Nutzer- und/oder Entwicklerdokumentation, Feedbackprogramme, etc.

- Standards
 - IEEE Std 828 (SCM Plans), ANSI-IEEE Std 1042 (SCM), ITIL, ...
- Bestandteile eines Plans
 - **Was** wird verwaltet (managed)? (listen und organisieren von CIs)
 - **Wer** ist für welche Aktivitäten verantwortlich? (Rollen und Aufgaben)
 - **Wie** macht man es? (Prozess für Änderungsanfragen, Aufgabenverteilung, Überwachung, Testen, Release, ...)
 - Was wird **aufgezeichnet**? (Logs, Konfigurationen, Änderungen, etc.)
 - **Welche** Ressourcen und welche **Kapazitäten** werden benötigt? (Werkzeuge, Personen, Geld, ...)
 - Mit welchen Metriken wird Fortschritt und **Erfolg** gemessen?

- Build Prozess: Automatische Generierung eines fertigen Anwendungsprogramms
- Code-Compilierung und Linken
- Tools:
 - make, Cmake, automake, Ant, Maven, Gradle, MSBuild, ...
- Eigener Prozess für komplexe Entwicklungsprojekte

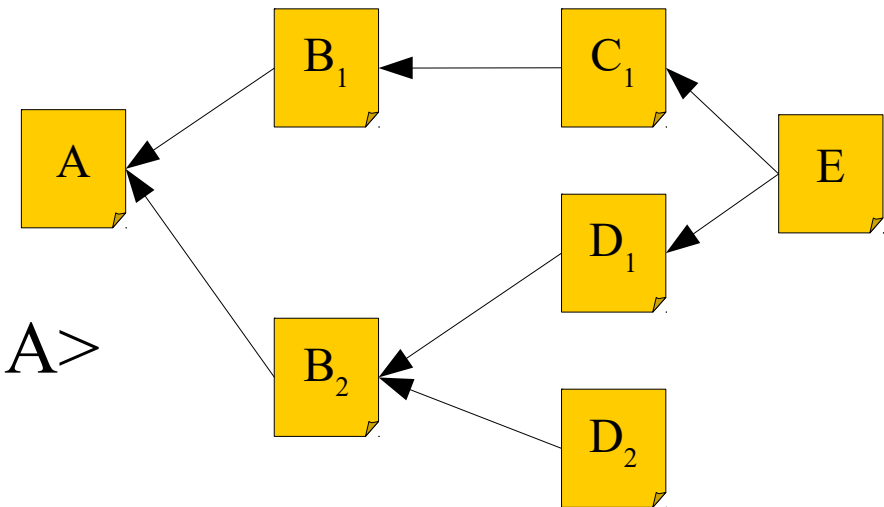
Wir betrachten hier die Diskussion um das Tool „make“:

- Rekursives Make
- Globale Abhängigkeiten der Module als gerichteter azyklischer Graph (DAG)
- $O(n)$ Updates notwendig (n =Anzahl der Module im System)

Beispiel: make A

Rekursive Tiefensuche ergibt

Buildreihenfolge $\langle C_1, B_1, D_1, D_2, B_2, A \rangle$



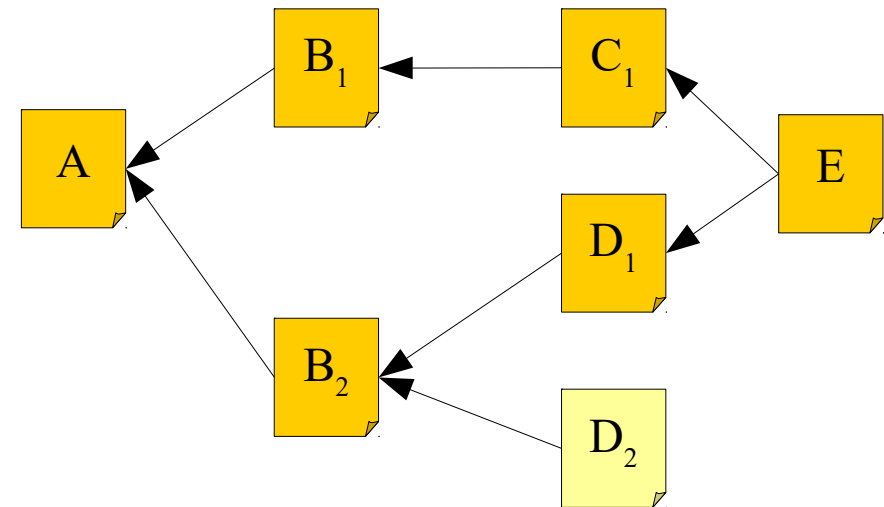
Pseudo-Algorithmus für rekursives Build:

- 1) Erzeuge DAG: Erzeuge globalen DAG aus Modulabhängigkeiten
- 2) Update: Durchlaufe DAG um zu prüfen, welche Module veraltet (out-of-date) sind und führe auf diesen Update Kommandos aus

```
update_file(f)
  need_update = false
  foreach dependency d {
    if(update_file(d) == UPDATED ||
       timestamp(d) newer than timestamp(f))
      need_update = true
  }
  if(need_update) {
    perform command to update f
    return UPDATED
  }
return PASS
```


Bei jedem Erstellen wird gesamter DAG durchlaufen und Module erstellt. Auch wenn nur eine Datei (z.B. D_2) geändert wurde.

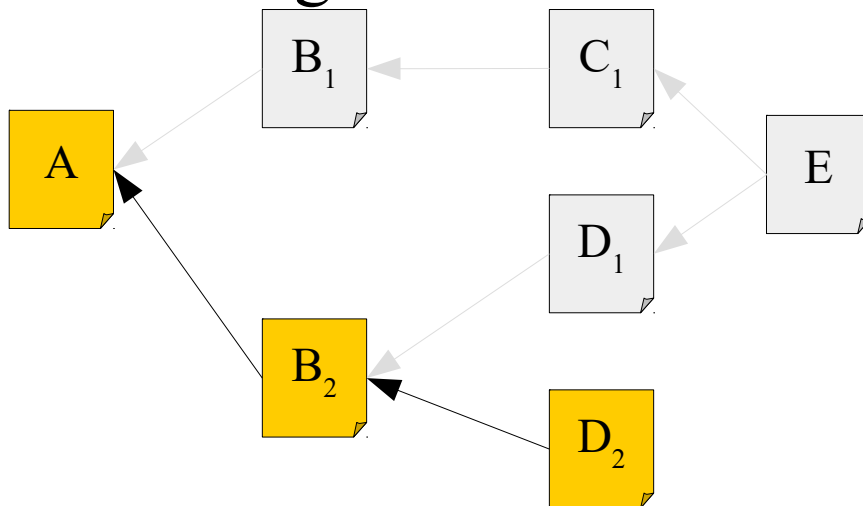
$O(n)$ Updates notwendig



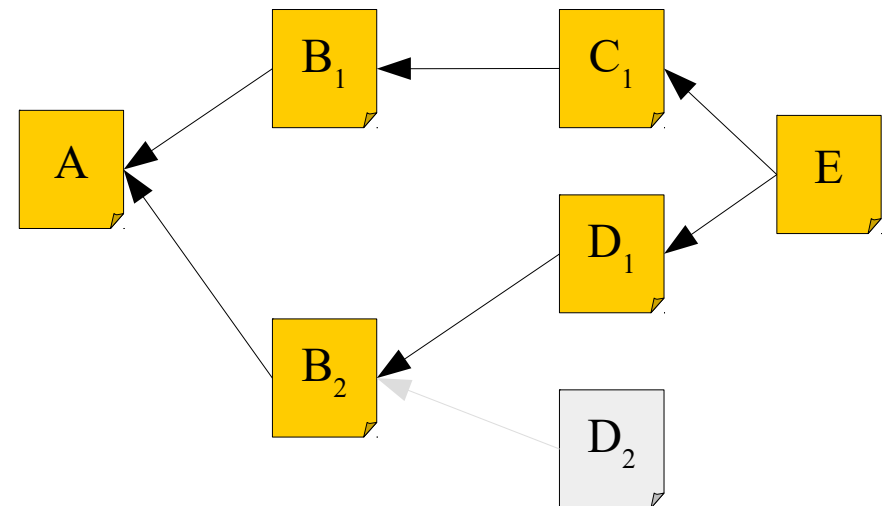
Anstatt kompletten DAG betrachten wir partiellen DAG der geänderten Dateien.

Def.: Partieller DAG

Ein partieller DAG ist Subgraph eines globalen DAGs, welcher genug Informationen über das Auffrischen des Systems während Konsistenz gewahrt bleibt.



Beispiel 1: Partieller DAG, der D_2 enthält



Beispiel 2: Partieller DAG, der E enthält

Pseudo-Algorithmus für Beta Buildsystem:

- 1) Lese Moduländerungsliste ein: Notwendige a priori information für Beta Buildsystem
- 2) Erzeuge partiellen DAG: Erzeuge einen partiellen DAG auf der Basis der Moduländerungsliste und einen vollständigen (globalen) DAG.
- 3) Update: Führe Updatekommandos aus, um alle Knoten im partiellen DAG aufzufrischen.

```
build_partial_DAG(DAG, change_list)
    foreach file_changed in change_list {
        add_node(DAG, file_changed)
    }

add_node(DAG, node)
    add node to DAG
    dependency_list = get_dependencies(node)
    foreach dependency d in dependency_list {
        if d is not in DAG {
            add_node(DAG, d) }
    add link (node -> d) to DAG
}
```

Update nun sehr einfach wegen der folgenden Eigenschaften:

- 1) Partieller DAG enthält nur Module, welche im System aufgefrischt werden müssen
- 2) Partieller DAG enthält keine Module, welche nicht aufgefrischt werden müssen.

```
update(DAG)
  file_list = topological_sort(DAG)
  foreach file in file_list {
    perform command to update file
  }
```

Komplexität der einzelnen Schritte:

- Modulabhängigkeiten einlesen: $O(1)$
- Partiellen DAG erstellen: $O(\log^2 n)$ (mit $\log^2 n = (\log n) * (\log n)$)
- Update: $O(\log n)$

Dominierender Faktor für das Verfahren ist $O(\log^2 n)$.

Automatisierung der unterschiedlichen Aufgaben eines Softwareentwicklers:

- Übersetzen des Quellcodes in binär Programm
- Verpacken des binär Codes (jar, war, etc.)
- Ausführung von Tests
- Bereitstellung des Produkts auf Produktionssystemen
- Erstellung der Dokumentation und Freigabevermerk (release note)
- ...

Build Management ist Prozess, der die einzelnen **Komponenten** einer Softwareanwendung in ein **installierbares Softwareprodukt zusammenfügt**.

Probleme:

- Compliance mit Regularien (SOX)
- Verteilte Entwicklungsteams
- Focus auf Wiederverwendbarkeit (SOA)
- Out-sourcing von Entwicklungsleistungen
- Marktdruck

Fortlaufendes Zusammenfügen von Komponenten zu einer Anwendung.

Ziel: Steigerung der Softwarequalität

Üblicherweise wird neben

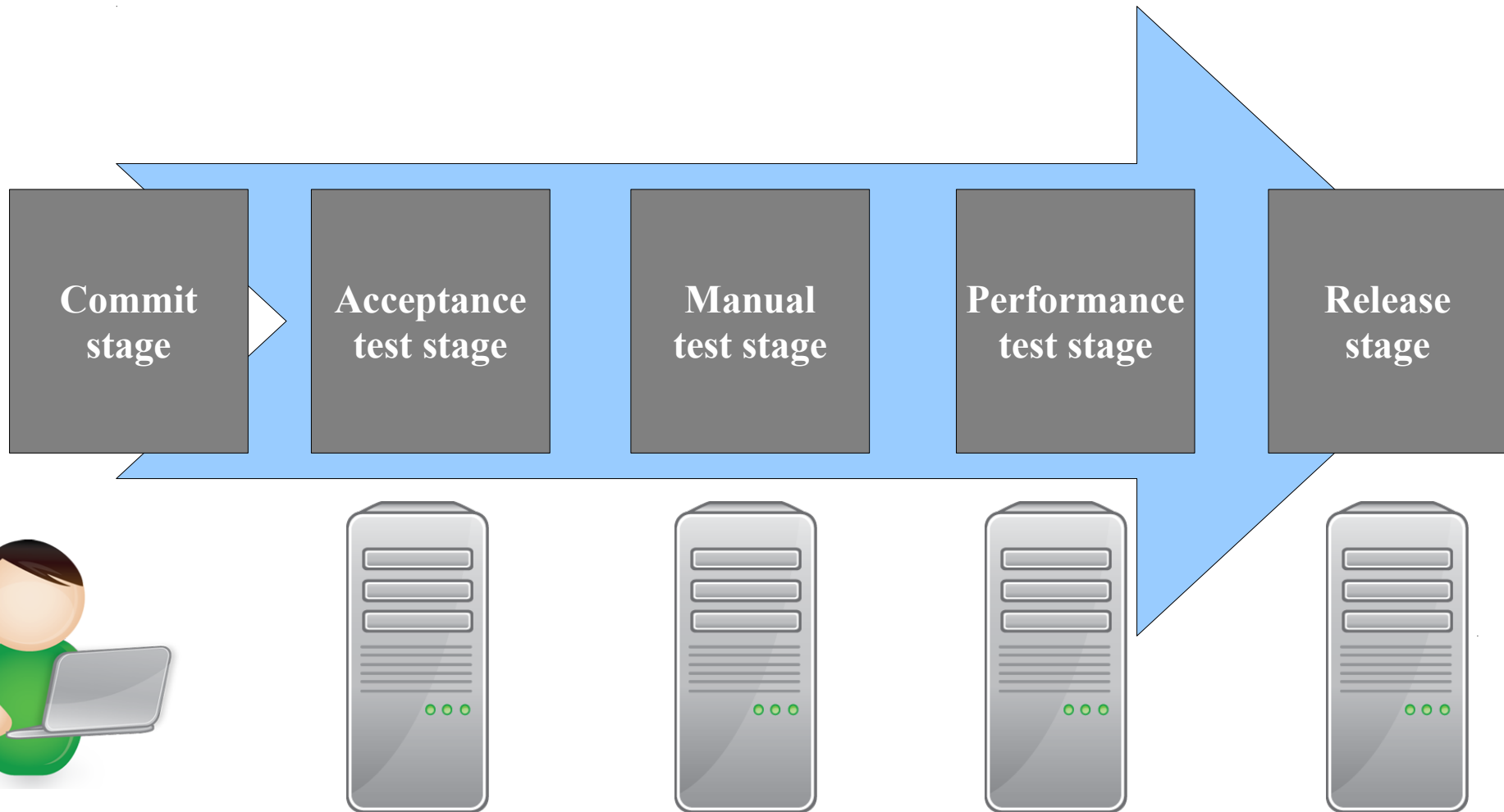
- Bau des Gesamtsystems auch
- automatisierte Tests durchgeführt sowie
- Metriken angewendet.

- Gemeinsame Codebasis
- Automatisierte Übersetzung
- Kontinuierliche Test-Entwicklung
- Häufige Integration
- Integration in den Hauptbranch
- Kurze Testzyklen
- Gespiegelte Produktionsumgebung
- Einfacher Zugriff
- Automatisiertes Reporting
- Automatisierte Verteilung

Häufigste Fehler beim Release (Release Antipatterns):

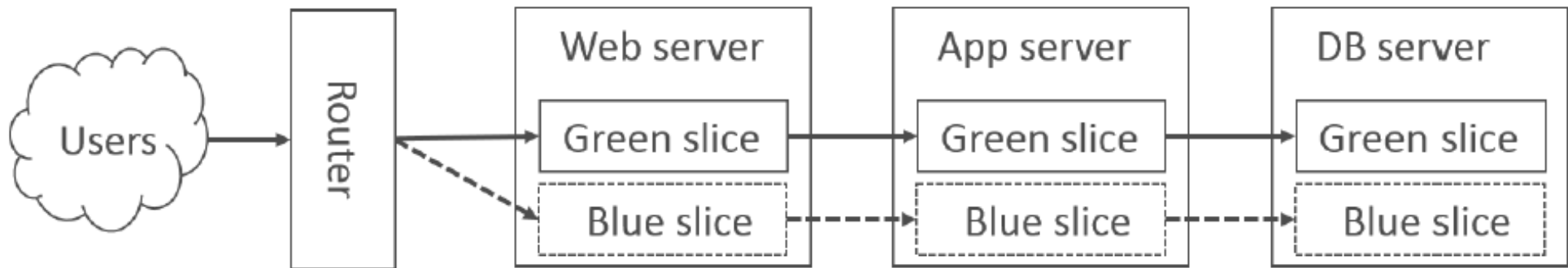
- **Manuelle** Software Deployments,
- die Software wird erst **nach ihrer Fertigstellung** in einer der **Produktivumgebung** ähnlichen Testumgebung **getestet**,
- die **manuelle** Konfiguration der Produktivumgebung.

Continuous Delivery = Continuous Integration + Release



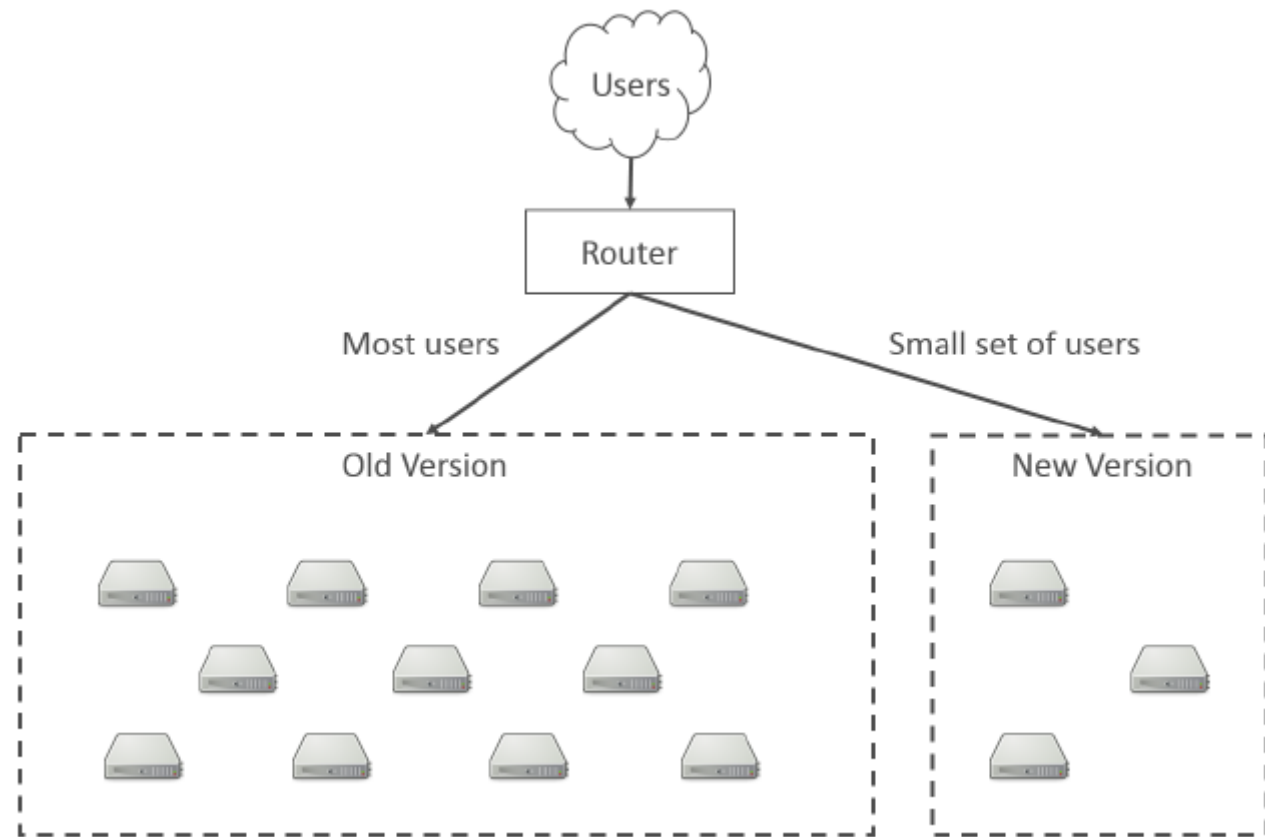
- **Commit Stage** wird Code kompiliert.
 - Durchführung von Unit Tests und Code Analyse.
 - Packen der Software in ein Format (jar, war, PE).
- **Acceptance Test Stage:** Test von User Stories.
 - Im Gegensatz zu Unit Tests werden hier Anwendungsfälle bzw. funktionale Eigenschaften getestet.
- **Manual Test Stage:** explorative Tests
- **Performance Test Stage:** Testen von nicht funktionalen Eigenschaften, wie Performanz, Skalierbarkeit, Durchsatz, Verfügbarkeit usw.
- **Release Stage:** Software auf Produktivsystem installiert oder als installierbare Software gepackt

- Blue-Green Deployments: identische Version der Produktivumgebung



- Bei Problem umschalten auf vorhergehendes Deployment
- Continuous deployment (Web Applikationen): Aktivieren und deaktivieren von Features (configs) über Flags

Kanarienvogel Release (canary release): Release auf Teilmenge der Produktivsysteme



- Softwarekonfigurationsmanagement ist zentraler Bestandteil der Softwareentwicklung
- Versionsmanagement verwaltet Konfigurationen
- Build Management steuert die Erstellung von Produkten
- Release Management verwaltet Erzeugung und Bereitstellung von neuen Softwareversionen für die Öffentlichkeit
- Continuous Delivery setzt Idee von Continuous Integration fort